

Lecture 4

IntLists (and Packages as Extra Content)

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin

CC BY NC SA

Finishing Lecture 4

We started today by finishing up the 2D array slides from Lecture 4: https://docs.google.com/presentation/d/1N01uiAblWZu5DePHTrelYfyuCd_Ck1QChgoNQhun1II/edit?slide=id.g3bbdc6aa9e7_0_568#slide=id.g3bbdc6aa9e7_0_568

CC BY NC SA

Packages (Extra Content) (Extra Content)

Lecture 4, CS61B, Fall 2026

CC BY NC SA

Packages (Extra Content) (Extra Content)

Goal: Build a List

IntLists

Packages (Extra Content)

Before we start today, let's talk briefly about Packages (Extra Content).

- A bit dull, but I hope this saves you from future trouble.

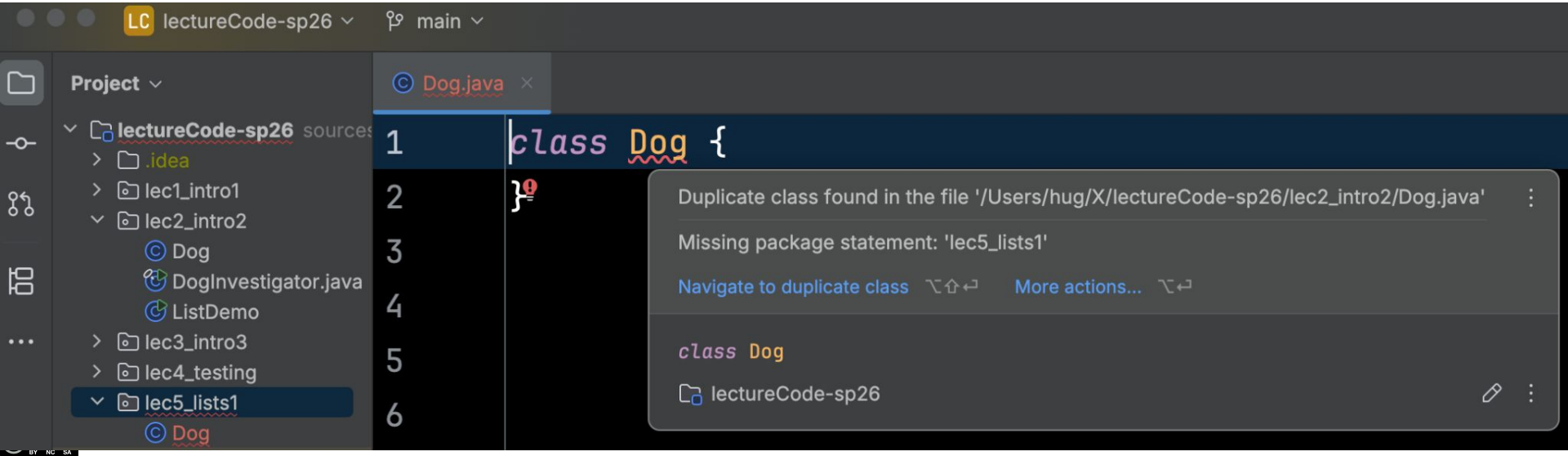
CC BY NC SA

Duplicate Classes

Suppose I:

- Have `Dog.java` in the `lec2_intro2` folder.
- Have `Dog.java` in the `lec5_lists1` folder.

Java will get mad, because it can see two Dog classes.



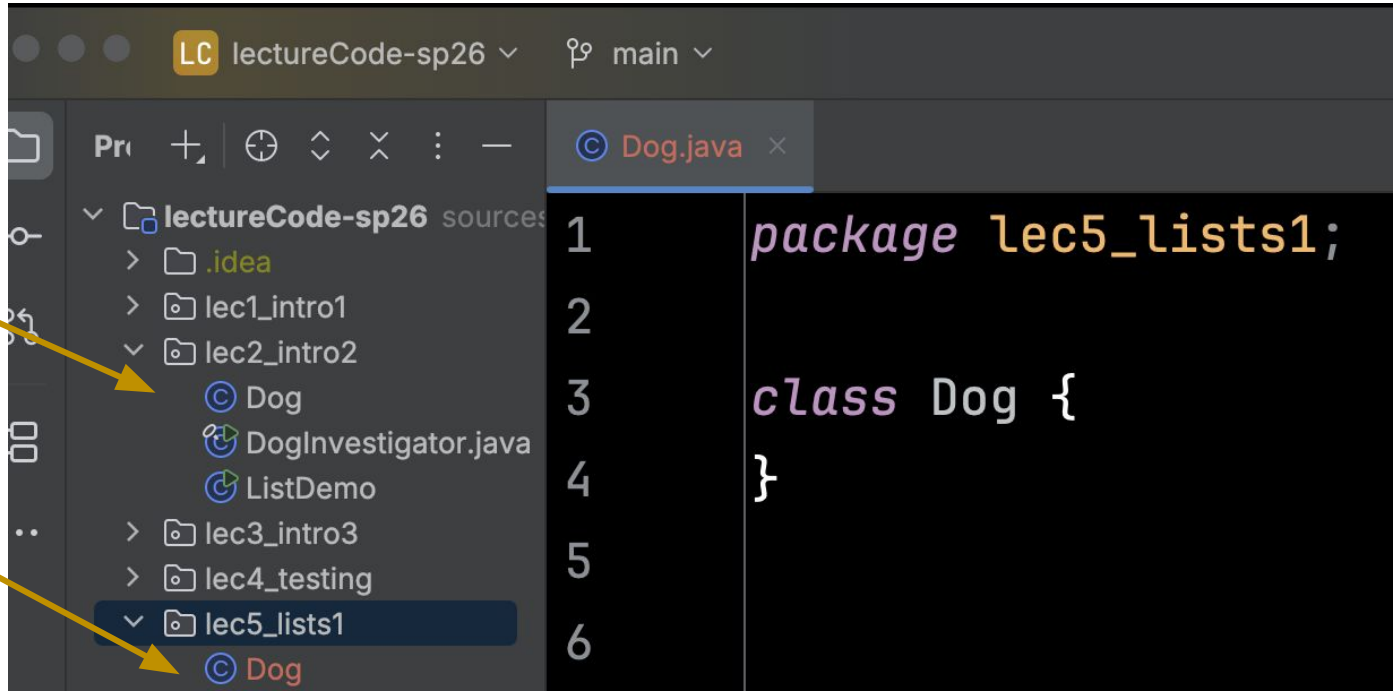
Declaring Code to be Part of a Package

By adding `package lec5_lists1;` to the first line of my file, I give a new "canonical" name to this Dog class.

- The one from `lec2_intro2` is just named `Dog` (because it has no package).
- This one is now named `lec5_lists1.Dog`.

`Canonical` name is `Dog`

`Canonical` name is `lec5_lists1.Dog`



CC BY NC SA

Importing (is not strictly necessary)

Importing is not required to use code from another package.

- If you don't import, you must give the full canonical name.
- For example, the code below is valid:

```
package lec5_lists1;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        lec4_testing.Sort.sort(input);
        com.google.common.truth.Truth.assertThat(input).isEqualTo(expected);
    }
}
```

full canonical name

CC BY NC SA

Importing Means “This is Shorthand for”

When you import a class, e.g. `lec4_testing.Sort`, you are saying:

- Whenever I write `Sort`, this is shorthand for `lec4_testing.Sort`.

```java
package lec5_lists1;
import lec4_testing.Sort;
import static com.google.common.truth.Truth.assertThat;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        Sort.sort(input);
        assertThat(input).isEqualTo(expected);
    }
}
```

CC BY NC SA

Importing (is not strictly necessary)

You can also import static members of a class, example below.

- `com.google.common.truth`: Package name.
- `Truth`: A class in the `com.google.common.truth` package:
- `assertThat`: A static method in the `Truth` class.

```
package lec5_lists1;
import lec4_testing.Sort;
import static com.google.common.truth.Truth.assertThat;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        Sort.sort(input);
        assertThat(input).isEqualTo(expected);
    }
}
```

CC BY NC SA

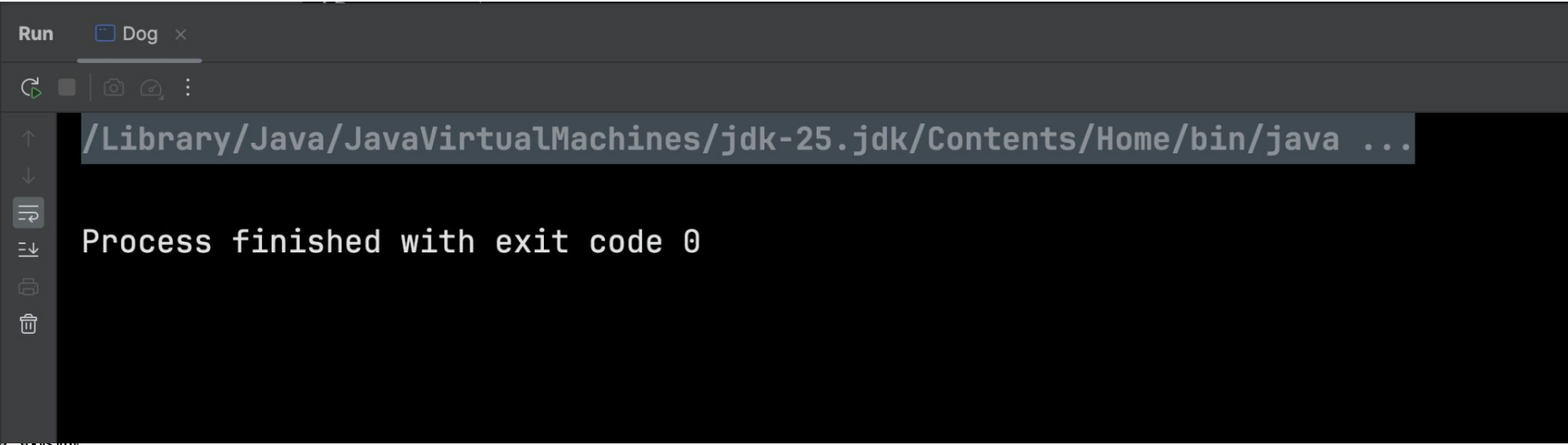
The CLASSPATH

Java won't look at every folder on your computer.

- Too slow, might pick up things you don't want to use.

Instead, it only looks on the CLASSPATH.

- To see your CLASSPATH, click the … in the terminal after running your code:



The CLASSPATH

Java won't look at every folder on your computer.

- Too slow, might pick up things you don't want to use.

Instead, it only looks on the CLASSPATH.

- To see your CLASSPATH, click the … in the terminal after running your code:

```
/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ
IDEA CE.app/Contents/lib/idea_rt.jar=63182 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
-Dsun.stderr.encoding=UTF-8 -classpath /Users/hug/X/lectureCode-sp26/out/production/lectureCode-sp26
:/Users/hug/X/library-fa25/ucb.jar:/Users/hug/X/library-fa25/algs4
.jar:/Users/hug/X/library-fa25/jh61b.jar:/Users/hug/X/library-fa25/proj2a
.jar:/Users/hug/X/library-fa25/asm-9.1.jar:/Users/hug/X/library-fa25/gson-2.10
```

CC BY NC SA

Public vs. Package-Private

In this class, we sometimes declared using `public`, and sometimes without. Difference:

- If you say `public`: Usable by code in other Packages (Extra Content).
- If you omit `public`: Usable only by code in the same package.

Dog.java

```
1  package lec5_lists1;
2
3  class Dog {
4  }
```

Only usable by other `lec5_lists1` code

Usable by the public, i.e. an class in any package

Sort.java

```
1  package lec4_testing;
2
3  public class Sort {
4      public static void sort(String[] input) {
5          sort(input, 0);
6      }
```

CC BY NC SA

Attendance

Meet the people sitting next to you!

- www.yellkey.com/expert



CC BY NC SA

Goal: Build a List

Lecture 4, CS61B, Fall 2026

Packages (Extra Content)

Goal: Build a List

IntLists

- Definition, size, iterativeSize()
- get()
- Destructive vs. Non-Destructive Methods

CC BY NC SA

Lists

Unlike Python, lists in Java aren't a core feature of the language.

```
import java.util.List;
import java.util.LinkedList;
List<String> L = new LinkedList<>();
L.add("a");
L.add("b");
```

Today, we'll begin our 4 lecture journey towards building our own list implementations.

- Will build two different types of lists: Linked List-based, then Array-based.

Today we'll cover the basic idea of Linked Lists. Then we'll take a break on Friday to cover automated code testing, before returning to lists next week.

CC BY NC SA

IntLists

Lecture 4, CS61B, Fall 2025

Packages (Extra Content)

Goal: Build a List

IntLists

- **Definition, size, iterativeSize()**
- get()
- Destructive vs. Non-Destructive Methods

CC BY NC SA

IntList

Let's define an `IntList` as an object containing two member variables:

- `int first;`
- `IntList rest;`

Then we'll see how you can add to the front and back of an `IntList`.

And define two versions of the same method:

- `size()`
- `iterativeSize()`

See the following hidden slides for the code we develop in class.

CC BY NC SA

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;









}
```

CC BY NC SA

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public static void main(String[] args) {
      IntList L = new IntList();
      L.first = 5;
      L.rest = null;









   }
}
```

CC BY NC SA

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public static void main(String[] args) {
      IntList L = new IntList();
      L.first = 5;
      L.rest = null;

      L.rest = new IntList();
      L.rest.first = 10;




   }
}
```

CC BY NC SA

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public static void main(String[] args) {
      IntList L = new IntList();
      L.first = 5;
      L.rest = null;

      L.rest = new IntList();
      L.rest.first = 10;

      L.rest.rest = new IntList();
      L.rest.rest.first = 15;

   }
}
```

Java Visualizer

CC BY NC SA

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;









}
```

CC BY NC SA

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }









}
```

CC BY NC SA

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }

   public static void main(String[] args) {
      IntList L = new IntList(15, null);
      L = new IntList(10, L);
      L = new IntList(5, L);



   }
}
```

Java Visualizer

CC BY NC SA

Coding Demo: IntList size

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }

   public static void main(String[] args) {
      IntList L = new IntList(15, null);
      L = new IntList(10, L);
      L = new IntList(5, L);

      System.out.println(L.size());  // should print out 3
   }
}
```

CC BY NC SA

Coding Demo: IntList size

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using... recursion! */
   public int size() {




   }




}
```

CC BY NC SA

Coding Demo: IntList size

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using... recursion! */
   public int size() {
      if (rest == null) {

      }

   }




}
```

CC BY NC SA

Coding Demo: IntList size

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using... recursion! */
   public int size() {
      if (rest == null) {
         return 1;
      }

   }









}
```

CC BY NC SA

Coding Demo: IntList size

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using... recursion! */
   public int size() {
      if (rest == null) {
         return 1;
      }
      return 1 + this.rest.size();
   }




}
```

Java Visualizer

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }

   public static void main(String[] args) {
      IntList L = new IntList(15, null);
      L = new IntList(10, L);
      L = new IntList(5, L);

      System.out.println(L.iterativeSize());  // should also print out 3
   }
}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {









   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;









   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;
      int totalSize = 0;









   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;
      int totalSize = 0;
      while (p != null) {


      }

   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;
      int totalSize = 0;
      while (p != null) {
         totalSize += 1;

      }

   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;
      int totalSize = 0;
      while (p != null) {
         totalSize += 1;
         p = p.rest;
      }

   }


}
```

CC BY NC SA

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the size of the list using no recursion! */
   public int iterativeSize() {
      IntList p = this;
      int totalSize = 0;
      while (p != null) {
         totalSize += 1;
         p = p.rest;
      }
      return totalSize;
   }


}
```

Java Visualizer

CC BY NC SA

IntLists

Lecture 4, CS61B, Fall 2025

Packages (Extra Content)

Goal: Build a List

IntLists

- Definition, size, iterativeSize()
- **get()**
- Destructive vs. Non-Destructive Methods

CC BY NC SA

Challenge

Write a method `int get(int i)` that returns the ith item in the list.

- For simplicity, OK to assume the item exists.
- Front item is the 0th item.

Ways to work:

- Paper (best)
- Laptop
  - hugcode.com/tiger
  - (or once that link stops working, hugcode.com/61b/lecs/lec4)
- In your head (worst)

```
L.get(0): 5
L.get(1): 10
```

```
public class IntList {
   public int first;
   public IntList rest;
   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }

   public int get(int i) { ... }

   /** Return the size of this IntList. */
   public int size() {
      if (rest == null) {
         return 1;
      }
      return 1 + this.rest.size();
   ...
```

CC BY NC SA

Coding Demo: IntList get

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   public IntList(int f, IntList r) {
      first = f;
      rest = r;
   }

   public static void main(String[] args) {
      IntList L = new IntList(15, null);
      L = new IntList(10, L);
      L = new IntList(5, L);

      System.out.println(L.get(1));  // should print out 10
   }
}
```

CC BY NC SA

Coding Demo: IntList get

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the ith item of this IntList. */
   public int get(int i) {




   }




}
```

CC BY NC SA

Coding Demo: IntList get

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the ith item of this IntList. */
   public int get(int i) {
      if (i == 0) {

      }

   }




}
```

CC BY NC SA

Coding Demo: IntList get

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the ith item of this IntList. */
   public int get(int i) {
      if (i == 0) {
         return first;
      }

   }




}
```

CC BY NC SA

Coding Demo: IntList get

IntList.java

```
public class IntList {
   public int first;
   public IntList rest;

   /** Return the ith item of this IntList. */
   public int get(int i) {
      if (i == 0) {
         return first;
      }
      return rest.get(i - 1);
   }




}
```

Java Visualizer

CC BY NC SA

Attendance Question:

What is your comfort level with recursive data structure code?

A. Very comfortable.
B. Comfortable.
C. Somewhat comfortable.
D. I have never done this



http://www.yellkey.com/listen

CC BY NC SA

IntLists

Lecture 4, CS61B, Fall 2025

Packages (Extra Content)

Goal: Build a List

IntLists

- Definition, size, iterativeSize()
- get()
- **Destructive vs. Non-Destructive Methods**

CC BY NC SA

Last Goal of the Day (Time Permitting)

Let's write some code that will operate on `IntLists`. We'll put our work in a new file called `IntListTools.java`.

First goal: `incrementRecursiveNonDestructive(IntList L, int x)`

- Returns a copy of L, where every element is incremented by x. Makes no changes to L.

CC BY NC SA

Last Goal of the Day (Time Permitting)

Let's write some code that will operate on `IntLists`. We'll put our work in a new file called `IntListTools.java`.

First goal: `incrementRecursiveNonDestructive(IntList L, int x)`

- Returns a copy of L, where every element is incremented by x. Makes no changes to L.

Second goal: `incrementRecursiveDestructive(IntList L, int x)`

- Returns a copy of L, where every element is incremented by x. Also changes L as a side-effect.

CC BY NC SA

Last Goal of the Day (Time Permitting)

Non-Destructive:

- A method is non-destructive if it does not modify the object.
  - Example: `public IntList incrementRecursive`**`NonDestructive`**`()` returns an incremented version of the current `IntList` without modifying current list.

Destructive

- A method is destructive if it is allowed to modify the object:
  - Example: `public IntList incrementRecursive`**`Destructive`**`()` returns an incremented version of the current `IntList`, and as a side effect modifies the current list.
  - Destructive methods might also return `void` (meaning nothing).

CC BY NC SA